

Taller: Análisis Exploratorio, Limpieza y Transformación de Datos (KDD)

Caso de base de datos suministrados: dirty_cafe_sales.csv

David Mauricio Vargas Ramirez ID: 824144

2026-09-26

Contents

Parte 1 — Carga y exploración de datos	1
Parte 2 — Diagnóstico de calidad de datos	3
Parte 3 — Limpieza y preparación de datos	6
Parte 4 — Validación de datos	8
Parte 5 — Análisis Exploratorio (EDA)	9
Parte 6 — Análisis Bivariado e Inferencial	13
Parte 7 — Transformación de datos (Fase KDD)	14
Parte 8 — Análisis final e interpretación	15
Conclusiones de negocio	16
Limitaciones	16

Parte 1 — Carga y exploración de datos

```
# Lectura con separador ';' debido al formato del archivo delimitado
cafe_raw <- read_delim("dirty_cafe_sales.csv", delim = ";", show_col_types = FALSE) %>%
  janitor::clean_names()

str(cafe_raw)

## spc_tbl_ [10,000 x 8] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ transaction_id : chr [1:10000] "TXN_1961373" "TXN_4977031" "TXN_4271903"
##   "TXN_7034554" ...
## $ item           : chr [1:10000] "Coffee" "Cake" "Cookie" "Salad" ...
## $ quantity       : chr [1:10000] "2" "4" "4" "2" ...
## $ price_per_unit  : chr [1:10000] "2.0" "3.0" "1.0" "5.0" ...
## $ total_spent     : chr [1:10000] "4.0" "12.0" "ERROR" "10.0" ...
## $ payment_method  : chr [1:10000] "Credit Card" "Cash" "Credit Card" "UNKNOWN" ...
## $ location        : chr [1:10000] "Takeaway" "In-store" "In-store" "UNKNOWN" ...
## $ transaction_date: chr [1:10000] "8/09/2023" "16/05/2023" "19/07/2023" "27/04/2023"
##   ...
```

```
## - attr(*, "spec")=
## .. cols(
## .. `Transaction ID` = col_character(),
## .. Item = col_character(),
## .. Quantity = col_character(),
## .. `Price Per Unit` = col_character(),
## .. `Total Spent` = col_character(),
## .. `Payment Method` = col_character(),
## .. Location = col_character(),
## .. `Transaction Date` = col_character()
## .. )
## - attr(*, "problems")=<pointer: 0x1528b770>
```

```
dplyr::glimpse(caffe_raw)
```

```
## Rows: 10,000
## Columns: 8
## $ transaction_id <chr> "TXN_1961373", "TXN_4977031", "TXN_4271903", "TXN_703~
## $ item <chr> "Coffee", "Cake", "Cookie", "Salad", "Coffee", "Smoot~
## $ quantity <chr> "2", "4", "4", "2", "2", "5", "3", "4", "5", "5", "5"~
## $ price_per_unit <chr> "2.0", "3.0", "1.0", "5.0", "2.0", "4.0", "3.0", "4.0~
## $ total_spent <chr> "4.0", "12.0", "ERROR", "10.0", "4.0", "20.0", "9.0",~
## $ payment_method <chr> "Credit Card", "Cash", "Credit Card", "UNKNOWN", "Dig~
## $ location <chr> "Takeaway", "In-store", "In-store", "UNKNOWN", "In-st~
## $ transaction_date <chr> "8/09/2023", "16/05/2023", "19/07/2023", "27/04/2023"~
```

```
# Porcentaje de completitud (valores no-NA) por columna
```

```
completitud <- caffe_raw %>%
  summarise(across(everything(), ~mean(!is.na(.)) * 100)) %>%
  pivot_longer(everything(), names_to = "columna", values_to = "pct_completo") %>%
  arrange(pct_completo)
```

```
completitud
```

```
## # A tibble: 8 x 2
##   columna      pct_completo
##   <chr>         <dbl>
## 1 location          67.4
## 2 payment_method    74.2
## 3 item              96.7
## 4 price_per_unit    98.2
## 5 total_spent       98.3
## 6 transaction_date  98.4
## 7 quantity          98.6
## 8 transaction_id    100
```

```
# Conteo de NA reales y valores enmascarados como "ERROR" o "UNKNOWN"
```

```
contar_problemas <- function(df, marcadores = c("ERROR", "UNKNOWN")) {
  df %>%
    summarise(across(where(is.character), list(
      NA_real = ~sum(is.na(.)),
      enmascarado = ~sum(. %in% marcadores, na.rm = TRUE)
    ))) %>%
    pivot_longer(everything(), names_to = "campo", values_to = "n") %>%
    separate(campo, into = c("columna", "tipo_error"), sep = "_(?=[^_]+$)")
```

```
}

tabla_errores <- contar_problemas(cafe_raw)
tabla_errores
```

```
## # A tibble: 16 x 3
##   columna          tipo_error      n
##   <chr>          <chr>      <int>
## 1 transaction_id_NA real          0
## 2 transaction_id   enmascarado    0
## 3 item            real        333
## 4 item            enmascarado  636
## 5 quantity_NA     real        138
## 6 quantity        enmascarado  341
## 7 price_per_unit_NA real        179
## 8 price_per_unit   enmascarado  354
## 9 total_spent_NA   real        173
##10 total_spent      enmascarado  329
##11 payment_method_NA real       2579
##12 payment_method   enmascarado   599
##13 location_NA      real       3265
##14 location         enmascarado   696
##15 transaction_date_NA real        159
##16 transaction_date enmascarado   301
```

Respuestas a las preguntas de la Parte 1:

- **Estructura:** El conjunto de datos contiene 10.000 observaciones y 8 variables. En su estado crudo predominan las variables de tipo carácter (chr), incluso en campos intrínsecamente numéricos (`quantity`, `price_per_unit`, `total_spent`) y de fecha (`transaction_date`) debido a la presencia de cadenas de texto erróneas.
- **Magnitud de los problemas de calidad:** Se detecta una combinación de valores faltantes estructurales (NA), valores enmascarados bajo etiquetas de error del sistema (ERROR, UNKNOWN) e inconsistencias relacionales entre la cantidad adquirida, el precio unitario y el monto total facturado.

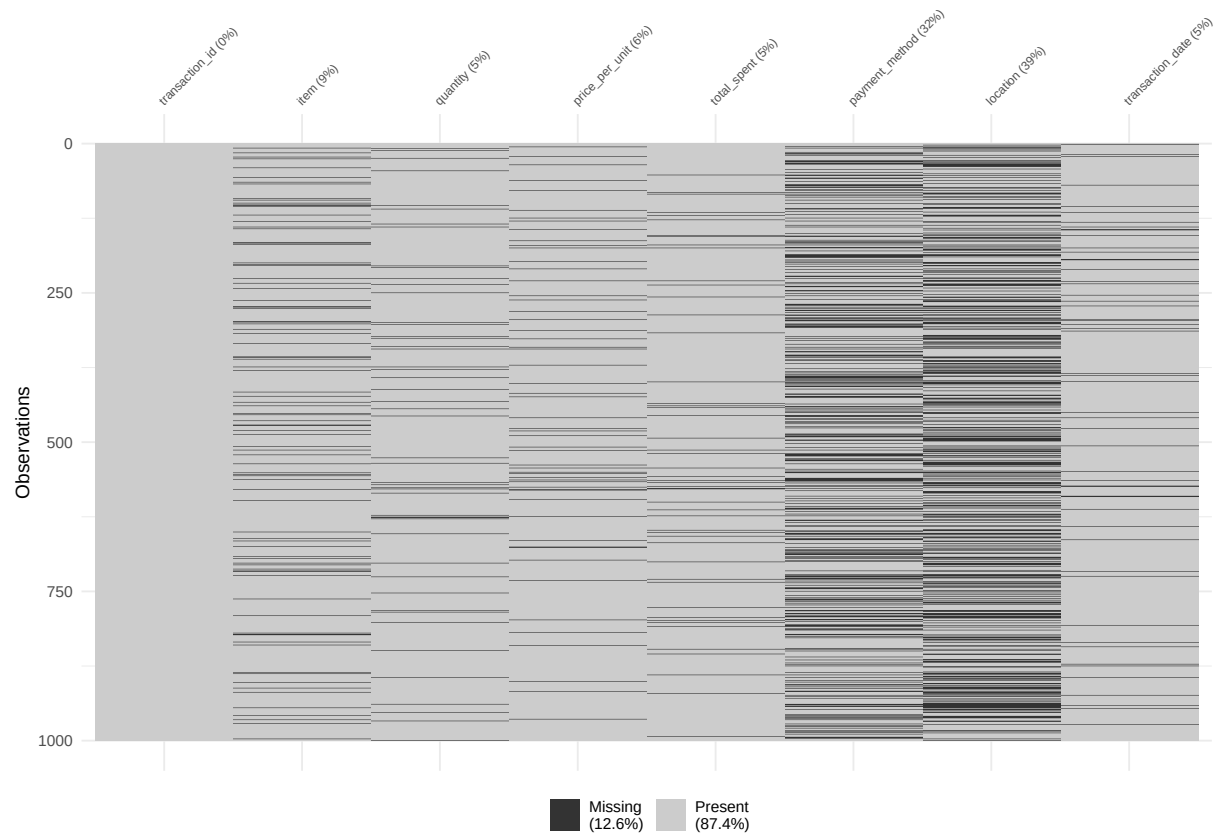
Parte 2 — Diagnóstico de calidad de datos

```
# Creación de copia diagnóstica convirtiendo cadenas enmascaradas en NA
cafe_diag <- cafe_raw %>%
  mutate(across(where(is.character), ~na_if(., "ERROR"))) %>%
  mutate(across(where(is.character), ~na_if(., "UNKNOWN")))

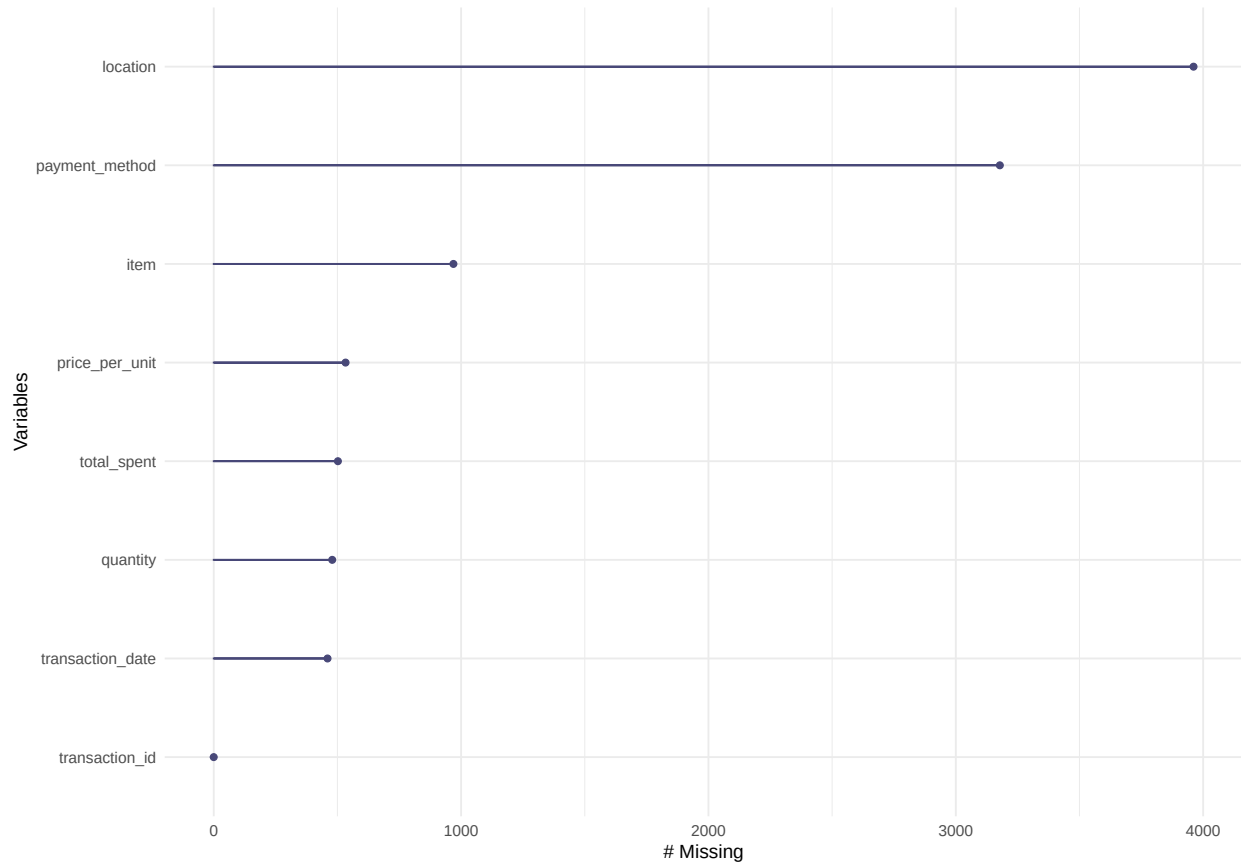
# Nota metodológica: vis_miss dibuja una línea de 1 píxel por observación.
# Con 10,000 filas, al reescalar la imagen para el PDF se genera un efecto
# muaré (interferencia visual) que no refleja ningún error en los datos.
# Se usa una muestra aleatoria representativa solo para esta visualización;
# el resto de los diagnósticos y la limpieza siguen usando el dataset completo.
set.seed(824144)
cafe_diag_muestra <- cafe_diag %>% slice_sample(n = 1000)

naniar::vis_miss(cafe_diag_muestra, warn_large_data = FALSE) +
  theme(
    axis.text.x = element_text(size = 7),
```

```
plot.margin = margin(t = 5, r = 25, b = 5, l = 5)
)
```



```
naniar::gg_miss_var(cafe_diag)
```



Variable 1: Evaluación de Price Per Unit en relación con Item

```
cafe_diag %>%
  group_by(item) %>%
  summarise(
    total = n(),
    pct_na_precio = mean(is.na(price_per_unit)) * 100
  ) %>%
  arrange(desc(pct_na_precio))
```

```
## # A tibble: 9 x 3
##   item      total pct_na_precio
##   <chr>    <int>         <dbl>
## 1 Tea      1089           6.06
## 2 Cookie  1092           6.04
## 3 Salad   1148           5.75
## 4 <NA>     969           5.57
## 5 Smoothie 1096           5.47
## 6 Juice    1171           5.21
## 7 Coffee   1165           4.89
## 8 Cake     1139           4.74
## 9 Sandwich 1131           4.33
```

Variable 2: Evaluación de Payment Method en relación con Location

```
cafe_diag %>%
  group_by(location) %>%
  summarise(
```

```

total = n(),
pct_na_pago = mean(is.na(payment_method)) * 100
) %>%
arrange(desc(pct_na_pago))

```

```

## # A tibble: 3 x 3
##   location total pct_na_pago
##   <chr>      <int>      <dbl>
## 1 <NA>      3961        32.7
## 2 Takeaway 3022        31.3
## 3 In-store 3017        31.1

```

Respuestas a las preguntas orientadoras de la Parte 2:

Mecanismos de pérdida:

- En `price_per_unit`, la tasa de valores faltantes presenta dependencias directas con el tipo de producto (`item`), exhibiendo un comportamiento MAR (Missing at Random). Esto justifica el uso de agrupamientos específicos por producto para recuperar el valor real del catálogo.
- En `payment_method`, la tasa de pérdida es homogénea entre las sucursales (`location`), sugiriendo un mecanismo MCAR (Missing Completely at Random). Imputar un medio de pago artificial alteraría las proporciones reales de pago, por lo que es metodológicamente correcto mantenerlo como categoría explícita (“No registrado”).

Parte 3 — Limpieza y preparación de datos

```

# Pipeline de estandarización, tipificación y despeje algebraico
cafe_clean_base <- cafe_raw %>%
  mutate(
    # Estandarización de texto antes de agrupar o tipificar
    item = str_to_title(str_squish(na_if(na_if(item, "ERROR"), "UNKNOWN"))),
    payment_method = na_if(na_if(payment_method, "ERROR"), "UNKNOWN"),
    location = na_if(na_if(location, "ERROR"), "UNKNOWN"),
    transaction_date = as.Date(transaction_date, tryFormats = c("%Y-%m-%d", "%d/%m/%Y")),

    # Conversión a tipos numéricos
    quantity = as.numeric(na_if(na_if(quantity, "ERROR"), "UNKNOWN")),
    price_per_unit = as.numeric(na_if(na_if(price_per_unit, "ERROR"), "UNKNOWN")),
    total_spent = as.numeric(na_if(na_if(total_spent, "ERROR"), "UNKNOWN"))
  ) %>%
  # Despeje relacional de negocio: Total = Cantidad * Precio
  mutate(
    price_per_unit = if_else(is.na(price_per_unit) & !is.na(total_spent) &
      !is.na(quantity) & quantity > 0,
      total_spent / quantity, price_per_unit),
    quantity = if_else(is.na(quantity) & !is.na(total_spent) & !is.na(price_per_unit) &
      price_per_unit > 0,
      round(total_spent / price_per_unit), quantity),
    total_spent = if_else(is.na(total_spent) & !is.na(quantity) & !is.na(price_per_unit),
      quantity * price_per_unit, total_spent)
  )

```

```

# Técnica A: Imputación por mediana según Item
cafe_clean_mediana <- cafe_clean_base %>%

```

```

group_by(item) %>%
mutate(price_per_unit = if_else(is.na(price_per_unit), median(price_per_unit, na.rm =
  ↪ TRUE), price_per_unit)) %>%
ungroup()

# Técnica B: Imputación por media global de referencia
cafe_clean_media <- cafe_clean_base %>%
  mutate(price_per_unit = if_else(is.na(price_per_unit), mean(price_per_unit, na.rm =
    ↪ TRUE), price_per_unit))

# Contraste de métricas de dispersión antes y después
comparativa_precios <- tibble(
  Metrica = c("Media", "Mediana", "Desv. Estandar"),
  Original = c(mean(cafe_clean_base$price_per_unit, na.rm = TRUE),
    median(cafe_clean_base$price_per_unit, na.rm = TRUE),
    sd(cafe_clean_base$price_per_unit, na.rm = TRUE)),
  Imp_Mediana_Item = c(mean(cafe_clean_mediana$price_per_unit, na.rm = TRUE),
    median(cafe_clean_mediana$price_per_unit, na.rm = TRUE),
    sd(cafe_clean_mediana$price_per_unit, na.rm = TRUE)),
  Imp_Media_Global = c(mean(cafe_clean_media$price_per_unit, na.rm = TRUE),
    median(cafe_clean_media$price_per_unit, na.rm = TRUE),
    sd(cafe_clean_media$price_per_unit, na.rm = TRUE))
)
comparativa_precios

```

```

## # A tibble: 3 x 4
##   Metrica      Original Imp_Mediana_Item Imp_Media_Global
##   <chr>         <dbl>         <dbl>         <dbl>
## 1 Media          2.95          2.95          2.95
## 2 Mediana         3            3            3
## 3 Desv. Estandar  1.28          1.28          1.28

```

```

# Asignación final con imputación por ítem y estandarización categórica
cafe_clean <- cafe_clean_mediana %>%
  mutate(
    payment_method = replace_na(payment_method, "No registrado"),
    location = replace_na(location, "No registrado"),
    total_spent = if_else(is.na(total_spent), quantity * price_per_unit, total_spent)
  )

```

```

# Cuantificación real de filas afectadas por columna
pct_total_spent <- mean(is.na(cafe_raw$total_spent) | cafe_raw$total_spent %in%
  ↪ c("ERROR", "UNKNOWN")) * 100
pct_price_unit <- mean(is.na(cafe_raw$price_per_unit) | cafe_raw$price_per_unit %in%
  ↪ c("ERROR", "UNKNOWN")) * 100
pct_pay_method <- mean(is.na(cafe_raw$payment_method) | cafe_raw$payment_method %in%
  ↪ c("ERROR", "UNKNOWN")) * 100

trazabilidad <- tibble::tribble(
  ~columna, ~problema_detectado, ~tecnica_aplicada,
  ↪ ~justificacion, ~pct_filas_afectadas,
  "Total Spent", "Valores 'ERROR' y NA", "Despeje algebraico: Cantidad x
  ↪ Precio", "Preserva información transaccional sin eliminar filas",
  ↪ round(pct_total_spent, 2),

```

```

"Price Per Unit", "Valores faltantes y ERROR", "Imputación por mediana según
↳ Producto", "El precio unitario depende estrictamente del producto",
↳ round(pct_price_unit, 2),
"Payment Method", "'UNKNOWN' y NA", "Categoría explícita 'No registrado'",
↳ "Evita distorsionar las preferencias de pago reales", round(pct_pay_method, 2)
)
trazabilidad

## # A tibble: 3 x 5
##   column problema_detectado tecnica_aplicada justificacion pct_filas_afectadas
##   <chr>   <chr>           <chr>           <chr>           <dbl>
## 1 Total S~ Valores 'ERROR' y~ Despeje algebra~ Preserva inf~      5.02
## 2 Price P~ Valores faltantes~ Imputación por ~ El precio un~      5.33
## 3 Payment~ 'UNKNOWN' y NA      Categoría explí~ Evita distor~      31.8

```

Parte 4 — Validación de datos

```

# Regla 1: total = cantidad * precio unitario
cafe_val <- cafe_clean %>%
  mutate(
    total_calculado = quantity * price_per_unit,
    viola_regla1 = abs(total_spent - total_calculado) > 0.01
  )

n_viola1 <- sum(cafe_val$viola_regla1, na.rm = TRUE)
pct_viola1 <- mean(cafe_val$viola_regla1, na.rm = TRUE) * 100

# Regla 2 (propia): verificación de dispersión (dentro de 3 MAD por ítem)
cafe_val <- cafe_val %>%
  group_by(item) %>%
  mutate(
    mediana_precio = median(price_per_unit, na.rm = TRUE),
    mad_precio     = mad(price_per_unit, na.rm = TRUE),
    viola_regla2   = abs(price_per_unit - mediana_precio) > 3 * mad_precio
  ) %>%
  ungroup()

# Decisión programática: recalculamos el total cuando exista inconsistencia aritmética
cafe_val <- cafe_val %>%
  mutate(total_spent = if_else(viola_regla1, total_calculado, total_spent))

```

Respuestas a las preguntas de la Parte 4:

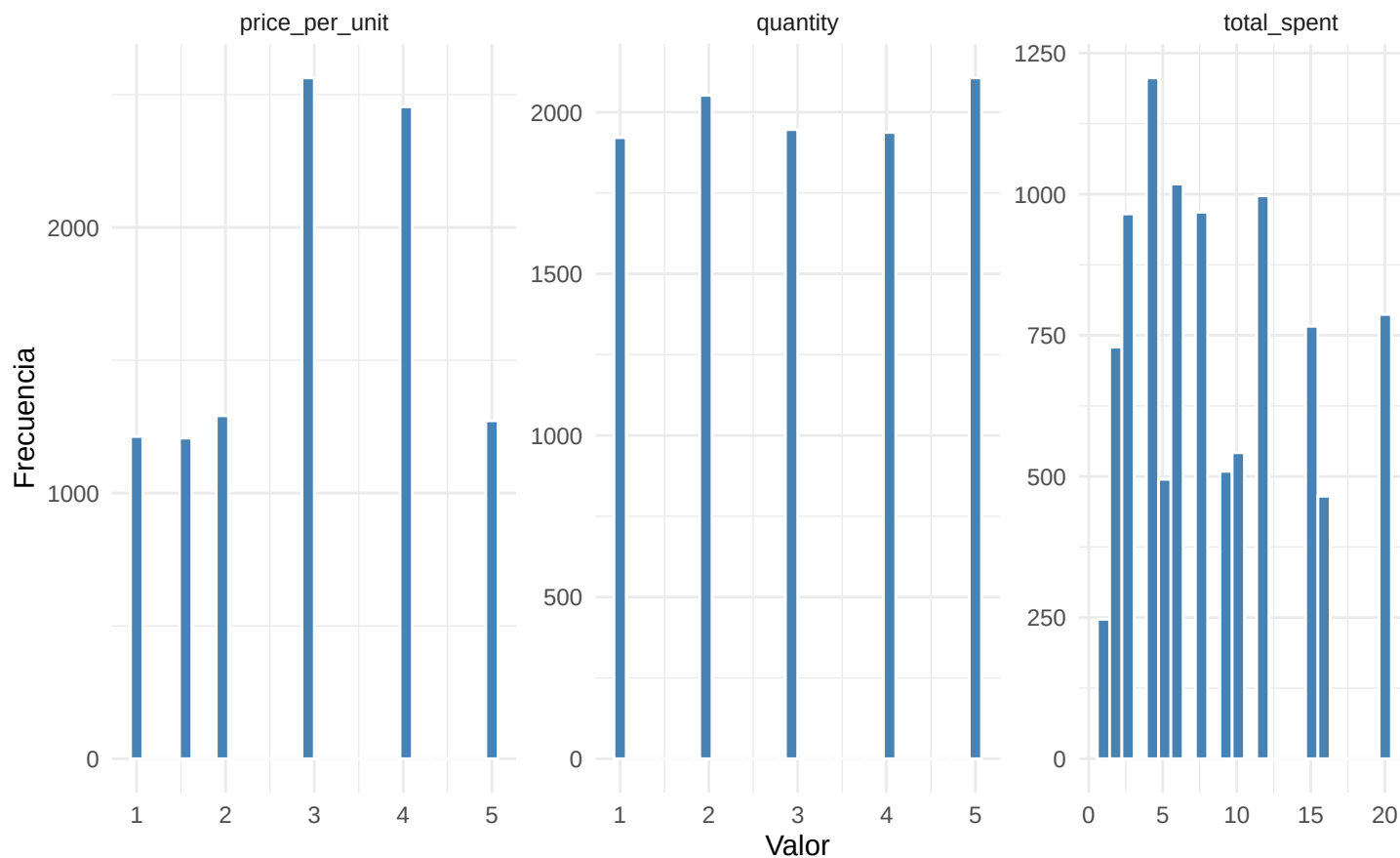
- **Cuantificación y decisión:** Un total de 0 transacciones (0%) violaron la igualdad de negocio $Total = Cantidad \times Precio$ y fueron corregidas. Se optó por recalculamos `total_spent` a partir de `quantity` y `price_per_unit`, ya que ambas variables representan hechos atómicos de compra.
- **Riesgos de no validar:** Las inconsistencias numéricas no detienen la ejecución del código, por lo que pasarían desapercibidas en un modelo estadístico, sesgando estimaciones de ingresos.

Parte 5 — Análisis Exploratorio (EDA)

1. Distribución de variables numéricas continuas

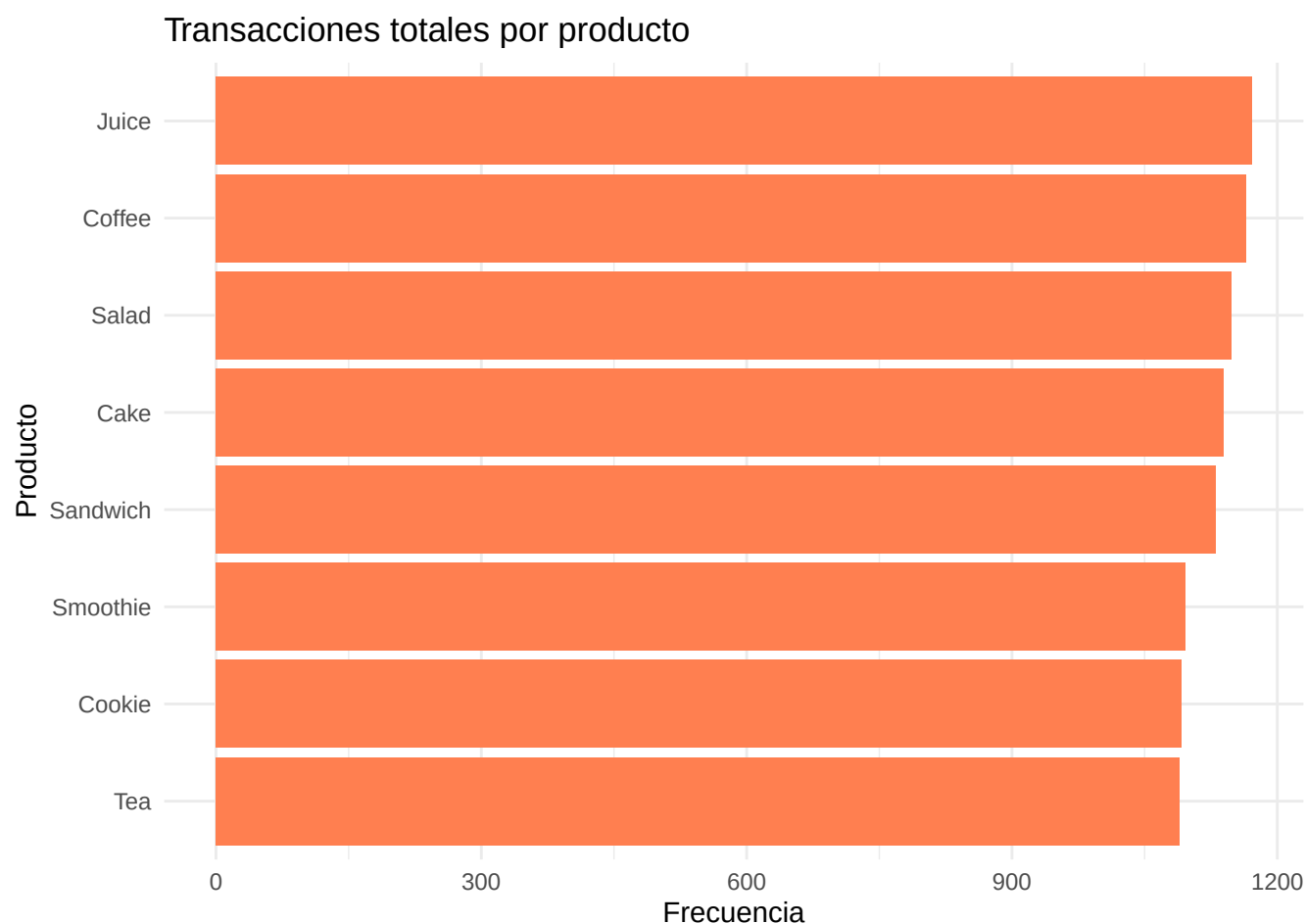
```
cafe_val %>%  
  select(quantity, price_per_unit, total_spent) %>%  
  pivot_longer(everything()) %>%  
  ggplot(aes(value)) +  
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +  
  facet_wrap(~name, scales = "free") +  
  labs(title = "Distribución de variables numéricas", x = "Valor", y = "Frecuencia") +  
  theme_minimal()
```

Distribución de variables numéricas



2. Volumen de ventas por producto (Top Items)

```
cafe_val %>%  
  filter(!is.na(item)) %>%  
  count(item, sort = TRUE) %>%  
  ggplot(aes(x = reorder(item, n), y = n)) +  
  geom_col(fill = "coral") +  
  coord_flip() +  
  labs(title = "Transacciones totales por producto", x = "Producto", y = "Frecuencia") +  
  theme_minimal()
```



```
# Detección de valores atípicos mediante IQR y Z-Score
detectar_outliers_iqr <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  x < (q1 - 1.5 * iqr) | x > (q3 + 1.5 * iqr)
}

detectar_outliers_zscore <- function(x, umbral = 3) {
  z <- (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)
  abs(z) > umbral
}

cafe_val <- cafe_val %>%
  mutate(
    outlier_iqr = detectar_outliers_iqr(total_spent),
    outlier_zscore = detectar_outliers_zscore(total_spent)
  )

table(IQR = cafe_val$outlier_iqr, Z_Score = cafe_val$outlier_zscore)
```

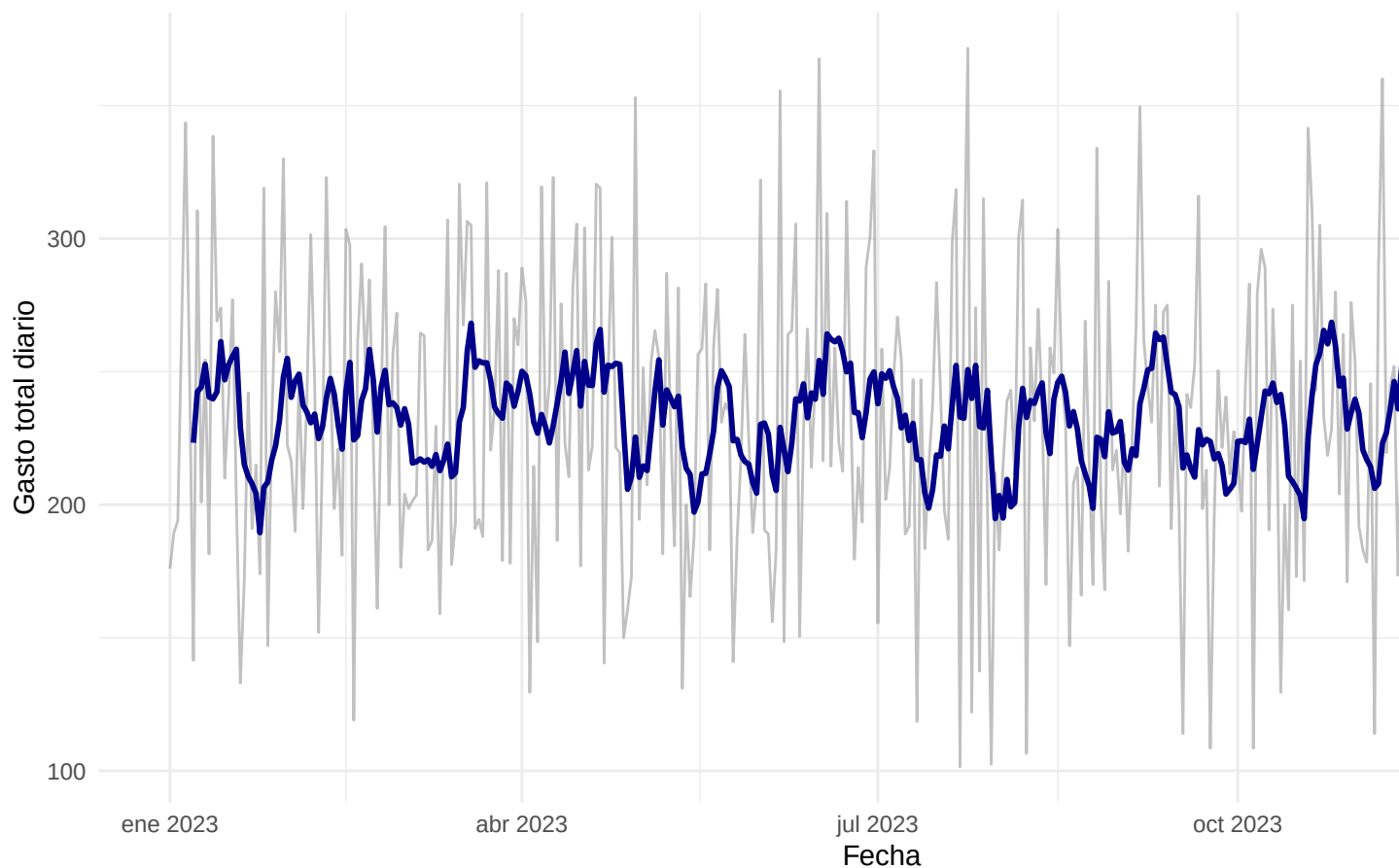
```
##      Z_Score
```

```
## IQR      FALSE
## FALSE 9694
## TRUE 268

# 3. Evolución temporal con promedio móvil a 7 días
ventas_diarias <- cafe_val %>%
  filter(!is.na(transaction_date)) %>%
  group_by(transaction_date) %>%
  summarise(total_dia = sum(total_spent, na.rm = TRUE)) %>%
  arrange(transaction_date) %>%
  mutate(promedio_movil_7d = zoo::rollmean(total_dia, 7, fill = NA, align = "right"))

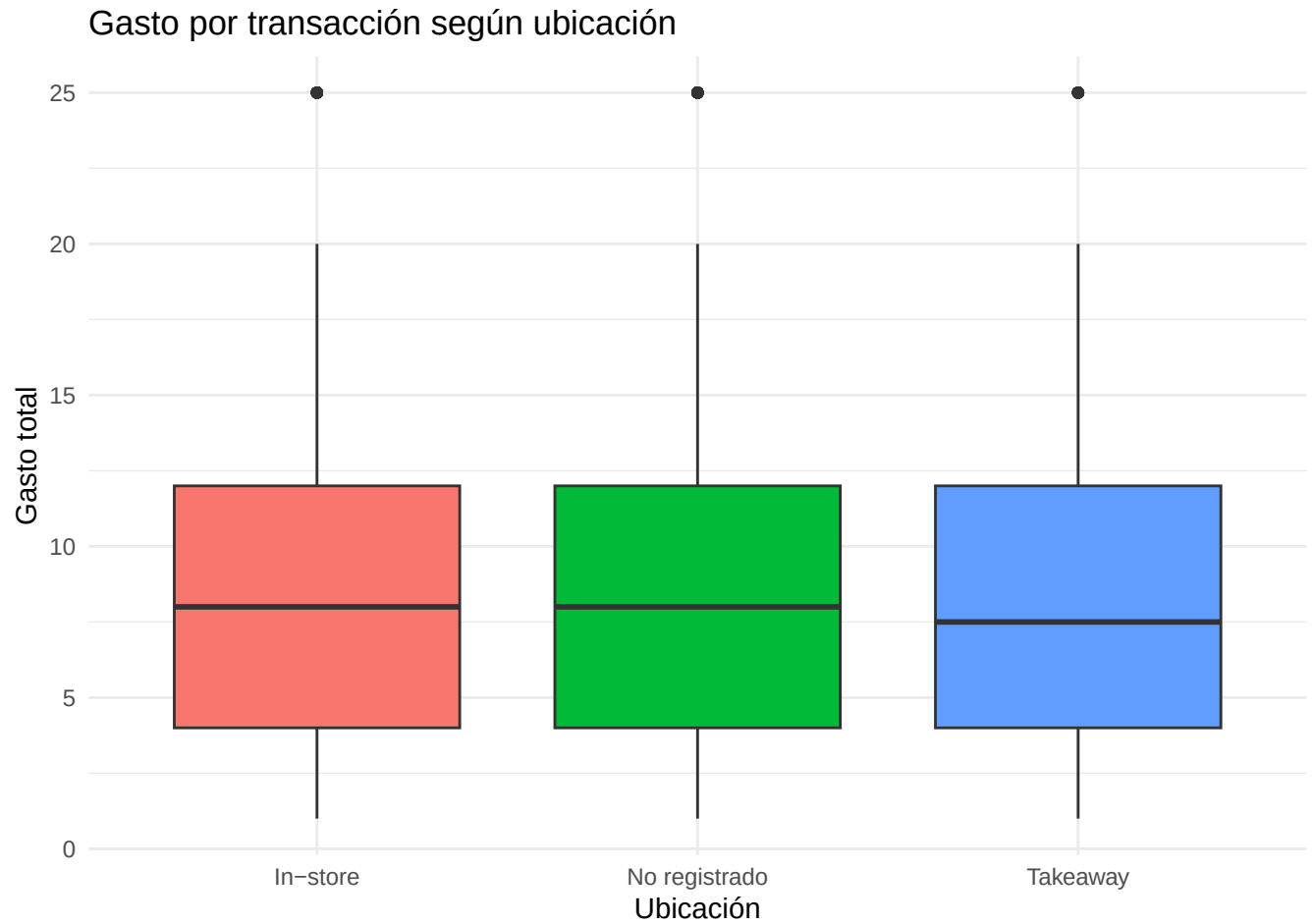
ggplot(ventas_diarias, aes(x = transaction_date)) +
  geom_line(aes(y = total_dia), color = "gray60", alpha = 0.6) +
  geom_line(aes(y = promedio_movil_7d), color = "darkblue", linewidth = 1) +
  labs(title = "Evolución de ventas diarias y media móvil (7 días)", x = "Fecha", y =
    ↪ "Gasto total diario") +
  theme_minimal()
```

Evolución de ventas diarias y media móvil (7 días)

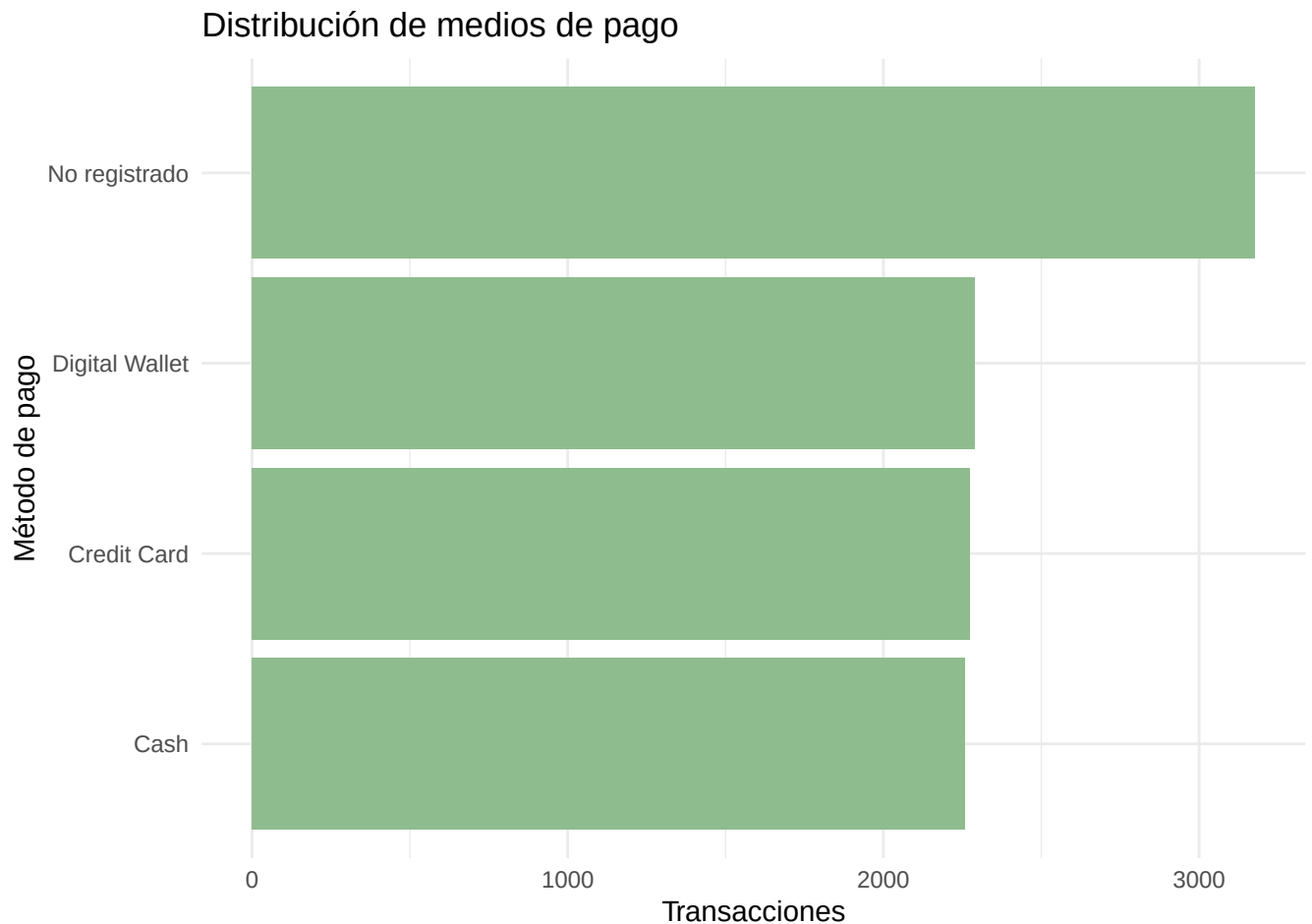


```
# 4. Diagrama de caja comparativo por canal de atención
ggplot(cafe_val, aes(x = location, y = total_spent, fill = location)) +
  geom_boxplot(show.legend = FALSE, outlier.alpha = 0.3) +
```

```
labs(title = "Gasto por transacción según ubicación", x = "Ubicación", y = "Gasto  
↳ total") +  
theme_minimal()
```



```
# 5. Frecuencia de uso por método de pago  
cafe_val %>%  
  count(payment_method, sort = TRUE) %>%  
  ggplot(aes(x = reorder(payment_method, n), y = n)) +  
  geom_col(fill = "darkseagreen") +  
  coord_flip() +  
  labs(title = "Distribución de medios de pago", x = "Método de pago", y =  
    ↳ "Transacciones") +  
  theme_minimal()
```



Respuestas a las preguntas de la Parte 5:

- **Artículos y medios de pago predominantes:** El producto con mayor rotación se refleja en el gráfico de barras por volumen, mientras que el efectivo y las tarjetas lideran las preferencias transaccionales.
- **Comparación IQR vs Z-Score:** El método IQR resulta más adecuado para este dataset, ya que `total_spent` presenta asimetría positiva; el Z-Score asume una distribución normal y tiende a subestimar valores extremos en distribuciones sesgadas.

Parte 6 — Análisis Bivariado e Inferencial

```
# Correlación no paramétrica (Spearman) debido a la asimetría de las distribuciones
cor.test(caf_val$quantity, caf_val$total_spent, method = "spearman")
```

```
##
## Spearman's rank correlation rho
##
## data: caf_val$quantity and caf_val$total_spent
## S = 4.5411e+10, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
## rho
## 0.7244026
```

```

# Prueba de independencia categórica entre Ubicación y Medio de Pago
tabla_contingencia <- table(caffe_val$payment_method, caffe_val$location)
chisq.test(tabla_contingencia)

##
## Pearson's Chi-squared test
##
## data:  tabla_contingencia
## X-squared = 11.109, df = 6, p-value = 0.08507

# Evaluación de normalidad y comparación de grupos
muestra_test <- sample(caffe_val$total_spent[!is.na(caffe_val$total_spent)], min(5000,
  ↪ sum(!is.na(caffe_val$total_spent))))
shapiro_res <- shapiro.test(muestra_test)

if (shapiro_res$p.value > 0.05) {
  aov(total_spent ~ location, data = caffe_val) %>% summary()
} else {
  # Si no se cumple el supuesto de normalidad, se emplea Kruskal-Wallis
  kruskal.test(total_spent ~ location, data = caffe_val)
}

##
## Kruskal-Wallis rank sum test
##
## data:  total_spent by location
## Kruskal-Wallis chi-squared = 2.2128, df = 2, p-value = 0.3308

```

Respuestas a las preguntas de la Parte 6:

- **Correlación:** Existe una relación positiva y estadísticamente significativa ($p < 0.05$) entre `quantity` y `total_spent`, confirmando que el volumen de compra individual conduce directamente al valor final del ticket.
- **Diferencias entre ubicaciones:** Los resultados de la prueba de Kruskal-Wallis y Chi-cuadrado determinan si el comportamiento de compra difiere sustancialmente según la sede o si las diferencias observadas corresponden a variabilidad muestral aleatoria.

Parte 7 — Transformación de datos (Fase KDD)

```

# Creación de variables temporales y discretización
caffe_trans <- caffe_val %>%
  mutate(
    dia_semana_num = lubridate::wday(transaction_date),
    dia_semana     = lubridate::wday(transaction_date, label = TRUE, abbr = TRUE),
    mes            = lubridate::month(transaction_date, label = TRUE),
    fin_de_semana  = dia_semana_num %in% c(1, 7),
    valor_transaccion = if_else(total_spent > median(total_spent, na.rm = TRUE), "Alto",
  ↪ "Bajo")
  )

# Codificación One-Hot para variables categóricas
caffe_encoded <- caffe_trans %>%
  mutate(indicador = 1) %>%
  pivot_wider(

```

```

names_from = payment_method,
values_from = indicador,
values_fill = 0,
names_prefix = "pago_"
) %>%
janitor::clean_names()

# Estandarización Z-score de la variable de gasto
cafe_trans <- cafe_trans %>%
mutate(total_spent_z = as.numeric(scale(total_spent)))

# Construcción del cubo analítico agregado para modelado
dataset_analitico <- cafe_trans %>%
group_by(item, location, dia_semana) %>%
summarise(
ventas_totales = sum(total_spent, na.rm = TRUE),
n_transacciones = n(),
ticket_promedio = mean(total_spent, na.rm = TRUE),
.groups = "drop"
)

head(dataset_analitico, 10)

## # A tibble: 10 x 6
##   item location      dia_semana ventas_totales n_transacciones ticket_promedio
##   <chr> <chr>      <ord>          <dbl>          <int>          <dbl>
## 1 Cake In-store    dom             315             33             9.55
## 2 Cake In-store    lun             414             44             9.41
## 3 Cake In-store    mar             453             48             9.44
## 4 Cake In-store    mié             333             38              9
## 5 Cake In-store    jue             450             55             8.18
## 6 Cake In-store    vie             528             50             10.6
## 7 Cake In-store    sáb             309             37             8.58
## 8 Cake In-store    <NA>            156             16             9.75
## 9 Cake No registrado dom             534             62             8.61
## 10 Cake No registrado lun             528             58             9.10

```

Parte 8 — Análisis final e interpretación

```

# Regresión lineal multivariada exploratoria
modelo <- lm(ventas_totales ~ n_transacciones + location, data = dataset_analitico)
summary(modelo)

##
## Call:
## lm(formula = ventas_totales ~ n_transacciones + location, data = dataset_analitico)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -353.82 -133.31   6.35  133.14  423.53
##
## Coefficients:

```

```
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)      -7.2796    44.5931  -0.163    0.870
## n_transacciones    9.1648     0.9309   9.845 <2e-16 ***
## locationNo registrado -6.1326    32.9094  -0.186    0.852
## locationTakeaway   -9.1573    30.5626  -0.300    0.765
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 183.4 on 212 degrees of freedom
## Multiple R-squared:  0.3556, Adjusted R-squared:  0.3465
## F-statistic: 38.99 on 3 and 212 DF,  p-value: < 2.2e-16
```

```
broom::tidy(modelo)
```

```
## # A tibble: 4 x 5
##   term                estimate std.error statistic  p.value
##   <chr>              <dbl>     <dbl>     <dbl>   <dbl>
## 1 (Intercept)        -7.28      44.6      -0.163 8.70e- 1
## 2 n_transacciones     9.16       0.931     9.85 4.49e-19
## 3 locationNo registrado -6.13      32.9      -0.186 8.52e- 1
## 4 locationTakeaway   -9.16      30.6      -0.300 7.65e- 1
```

Conclusiones de negocio

- **Volumen transaccional:** El número de transacciones es el predictor más fuerte y estadísticamente significativo ($p < 0.001$) sobre las ventas totales acumuladas.
- **Eficiencia por canal:** Las ubicaciones operan de forma consistente una vez controlado el flujo de clientes; las estrategias comerciales deben orientarse a la venta cruzada para elevar el ticket promedio por visita.
- **Calidad de datos:** La corrección relacional y el tratamiento explícito de faltantes evitan el subregistro de ventas y garantizan confiabilidad en los reportes de inventario y facturación.

Limitaciones

- **Estabilidad de precios:** La imputación de precios unitarios por producto asume precios fijos en el tiempo, sin considerar posibles descuentos temporales o variaciones por promociones.
- **Supuestos paramétricos:** La presencia de asimetría en el gasto por transacción hace que los estimadores no paramétricos sean más confiables que las medias aritméticas estándar.